🎓 **Exam Type: Practical**
⏳ **Duration: 6 Hours**

"Credit Card Fraud Detection — Imbalanced Classification with Threshold Optimisation"

Project Overview

You are working as a Junior Data Scientist at the fraud analytics team of a digital payments company similar to Paytm or Razorpay. Every day, millions of transactions flow through the platform, and a small fraction are fraudulent. Missing even a handful of fraud cases costs the company lakhs of rupees and damages customer trust.

You are given the Credit Card Fraud Detection dataset from Kaggle — one of the most extreme class imbalance benchmarks in industry ML, with only 0.17% of transactions being fraudulent. Your task is to build a robust fraud detection pipeline, handle the severe imbalance, and critically — go beyond default 0.5 classification thresholds to optimise the decision boundary for maximum business value.

📁 **Dataset: Credit Card Fraud Detection**

🔗 Kaggle Link: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

📊 Rows: 284,807 transactions | Features: 30 (V1–V28 are PCA-transformed, plus Time, Amount, Class)

🎯 Target Column: Class (1 = Fraud, 0 = Legitimate) — extreme class imbalance: 0.17% fraud

⚠️ Note: Sample 50,000 rows (stratified) for local execution. Keep the full fraud class intact.

🎯 Learning Objectives

By completing this exam, students will demonstrate:

- Understanding of extreme class imbalance and why accuracy is a misleading metric for fraud detection.
- Hands-on application of undersampling, oversampling (SMOTE), and class weighting strategies.
- Building Logistic Regression, Random Forest, and XGBoost classifiers for imbalanced data.
- Precision-Recall curve analysis and threshold tuning beyond the default 0.5 cutoff.
- Computing a cost-benefit analysis using a confusion matrix with real monetary values.
- Communicating model results to a non-technical business stakeholder.

🧠 Step 1: Problem Framing & Theory Notes

Write the following as Markdown cells in your notebook before coding:

1. Why is Accuracy a terrible metric for fraud detection? If 99.83% of transactions are legitimate and your model predicts everything as 'Not Fraud', what accuracy does it achieve? What is the Precision and Recall for the Fraud class in that case?
2. What is the Precision-Recall tradeoff? Draw a simple 2x2 confusion matrix and explain what happens when you lower the classification threshold from 0.5 to 0.2 for the Fraud class.
3. Explain 3 strategies to handle extreme class imbalance: (a) Random Undersampling, (b) SMOTE Oversampling, (c) class_weight='balanced'. What are the pros and cons of each?
4. What is the Precision-Recall AUC (PR-AUC)? Why is it preferred over ROC-AUC for extreme imbalance datasets?
5. In a fraud detection context, which is more costly: a False Positive (flagging a legit transaction as fraud) or a False Negative (missing real fraud)? Give a specific example of the business impact of each.

💡 Tip: This exam is as much about business reasoning as coding. A student who blindly uses accuracy to pick a model will lose significant marks in Steps 6 and 7.

📂 Step 2: Dataset Loading & EDA

2.1 Load & Sample

- Load creditcard.csv. Print .shape and check class distribution using value_counts(normalize=True).
- Create a stratified sample of 50,000 rows using train_test_split with stratify=y, random_state=42 (use just the first split). Confirm fraud ratio is preserved in the sample.
- Print .describe() on the sampled data. Focus on Time and Amount columns.

2.2 Target & Feature Analysis

- Plot a countplot of the Class column — use a log scale on the y-axis to visualise the imbalance clearly.
- Plot the distribution of Amount separately for Fraud vs Legitimate transactions (use overlaid histograms or kde plots). Do fraudulent transactions tend to be smaller or larger?
- Plot the distribution of Time for Fraud vs Legitimate. Is fraud more likely at certain times of day?
- Plot a heatmap of correlations between V1–V10 and Class. Which V-features are most correlated with fraud?

📌 Key insight to investigate: Fraudulent transactions tend to have smaller Amount values and cluster at specific time windows — does your EDA confirm or contradict this?

🔧 Step 3: Data Preprocessing & Feature Engineering

3.1 Feature Engineering

- Create Amount_log = np.log1p(Amount) — the Amount distribution is heavily right-skewed.
- Create Hour = (Time % 86400) // 3600 — extract the hour of day from the Time column (seconds since first transaction).
- Drop the original Time and Amount columns after creating the above features.

3.2 Scaling

- Apply StandardScaler to Amount_log and Hour only — V1–V28 are already PCA-scaled.

3.3 Train-Test Split

- Split: 80% train / 20% test with stratify=y, random_state=42.
- Print class distribution in both train and test sets. Confirm stratification preserved the fraud ratio.

3.4 Imbalance Handling — Three Strategies

Prepare three versions of the training set:

- Original (imbalanced): X_train, y_train — no modification.
- SMOTE: Apply SMOTE(random_state=42, sampling_strategy=0.1) to X_train — bring fraud up to 10% of majority class. Name: X_smote, y_smote.
- Undersampled: Use RandomUnderSampler(random_state=42, sampling_strategy=0.1) on X_train. Name: X_under, y_under.
- Print class counts for all three versions.

⚠️ Critical Rule: The test set (X_test, y_test) must stay completely untouched throughout — no resampling, no modification. All three strategies are applied to training data only.

🤖 Step 4: Model Building — Logistic Regression & Random Forest

4.1 Logistic Regression — Three Variants

Train Logistic Regression three times, one per training set version:

- LR on Original data (class_weight='balanced', max_iter=1000, random_state=42).
- LR on SMOTE data (class_weight=None, max_iter=1000, random_state=42).
- LR on Undersampled data (class_weight=None, max_iter=1000, random_state=42).
- For each variant — predict on X_test and report: Precision, Recall, F1 for the Fraud class, and PR-AUC.
- Print a side-by-side comparison table of all three LR variants. Which imbalance strategy worked best for LR?

4.2 Random Forest — Best Imbalance Strategy

- Using the best imbalance strategy identified above, train RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42, n_jobs=-1).
- Report: Precision, Recall, F1 (Fraud class), PR-AUC.
- Plot the Precision-Recall curve (not ROC — PR curve is more informative here).
- Plot feature importances — top 15 features. Which V-features are most predictive of fraud?

⚡ Step 5: Model Building — XGBoost with Full Tuning

5.1 XGBoost Baseline

- Train XGBClassifier(n_estimators=200, learning_rate=0.1, max_depth=4, scale_pos_weight=ratio, random_state=42) where ratio = count(non-fraud) / count(fraud) in original training data.
- The scale_pos_weight parameter is XGBoost's built-in class weighting — calculate and set it correctly.
- Report: Precision, Recall, F1 (Fraud class), PR-AUC. Plot Precision-Recall curve.

5.2 Hyperparameter Tuning

- Use RandomizedSearchCV (n_iter=15, cv=3, scoring='average_precision', random_state=42) on XGBoost.
- Tune: n_estimators [100,200,300], max_depth [3,4,5,6], learning_rate [0.01,0.05,0.1], subsample [0.6,0.8,1.0], colsample_bytree [0.6,0.8,1.0].
- Print best parameters and best CV PR-AUC score.
- Re-evaluate tuned XGBoost on X_test. Compare PR-AUC before and after tuning.

5.3 Threshold Optimisation on Best Model

Your best model outputs predicted probabilities — the default cutoff of 0.5 is rarely optimal for fraud detection.

- Use predict_proba on X_test to get fraud probabilities.
- Plot the Precision-Recall curve and mark the default threshold (0.5) on it.
- Use precision_recall_curve to scan all possible thresholds. Find the threshold that maximises F1-Score for the Fraud class.

- Also find the threshold that achieves Recall >= 0.90 with the highest possible Precision.
- Re-evaluate your best model at both optimal thresholds and compare all metrics.

💡 Real-world insight: In production fraud systems at Paytm or Razorpay, the threshold is a business decision — not a model decision. A risk officer might demand Recall >= 0.95 even at the cost of more false alarms. Your job is to show what that tradeoff looks like.

📈 Step 6: Model Evaluation, Threshold Tuning & Comparison

Create a final comparison table across all models and key threshold variants:

- Logistic Regression (best imbalance strategy, default threshold)
- Random Forest (default threshold)
- XGBoost Baseline (default threshold)
- XGBoost Tuned (default threshold 0.5)
- XGBoost Tuned (F1-optimal threshold)
- XGBoost Tuned (Recall >= 0.90 threshold)

For each row report: Precision (Fraud), Recall (Fraud), F1 (Fraud), PR-AUC, Threshold used.

- Plot all Precision-Recall curves on a single figure with a legend.
- Write a recommendation (4–6 sentences): Which model and threshold would you deploy at Paytm's fraud team? Justify using specific metric values from your table.

💰 Step 7: Business Simulation & Cost-Benefit Analysis

This step separates a data scientist from a business analyst. Use your best model at the F1-optimal threshold.

7.1 Assign Transaction Costs

- Assume: Average fraudulent transaction value = ₹4,500.
- Assume: Cost of investigating a flagged transaction (False Positive) = ₹150 (analyst time).
- Using your confusion matrix on X_test, calculate:
  - Money saved = TP × ₹4,500 (fraud caught)
  - Investigation cost = (TP + FP) × ₹150 (all flagged transactions reviewed)
  - Money lost = FN × ₹4,500 (fraud missed)
  - Net benefit = Money saved − Investigation cost

7.2 Threshold Sensitivity

- Repeat the above calculation for 5 different thresholds: 0.1, 0.2, 0.3, 0.5, your F1-optimal threshold.
- Present results as a table with columns: Threshold, TP, FP, FN, Money Saved, Investigation Cost, Money Lost, Net Benefit.
- Which threshold gives the highest Net Benefit? Is it the same as the F1-optimal threshold?
- Write 3 sentences interpreting the result in plain English for a Paytm risk manager who does not know what F1-Score means.

💡 Key insight: The business-optimal threshold is not always the F1-optimal threshold. This is a core real-world lesson — ML metrics and business metrics can diverge.

🚀 Step 8: Pipeline, Deployment & GitHub Submission

8.1 Save the Final Pipeline

- Wrap best model + scaler into a sklearn Pipeline. Include the optimal threshold as a model attribute or note it in the README.
- Save: joblib.dump(pipeline, 'fraud_detection_model.pkl').
- Test: load the pickle and run predictions on 10 sample transactions. Show predicted probability and final label at your chosen threshold.

8.2 GitHub Submission

- Create a GitHub repository named: credit-fraud-detection-supervised-learning.
- Push the following:
  - FraudDetection_SupervisedLearning.ipynb (fully executed notebook)
  - fraud_detection_model.pkl (saved pipeline)
  - summary_report.md (see below)
  - requirements.txt
  - README.md (project overview, dataset link, optimal threshold used, how to run)

8.3 Summary Report (summary_report.md)

Write ~400–500 words covering:

- Business problem and why standard accuracy fails here.
- Imbalance handling strategy chosen and why.
- Which model and threshold you recommend and why.
- Cost-benefit result — how much net benefit does your model create per 50,000 transactions?
- What would you improve in a production deployment (real-time scoring, model drift monitoring, feedback loop)?

Marking Scheme (50 Marks)

Total Marks: 50 — distributed across 3 components as follows:

Component	Marks	Deliverable
A. Technical Completion	30 Marks	Completed .ipynb & .pkl file with all tasks built and functional
B. Recorded Video (Face + Screen)	15 Marks	5-10 min MP4/MOV showing face + screen with verbal explanation
C. GitHub Repository + README.md	5 Marks	Public GitHub repo with screenshots, README, video link, commits
TOTAL	50 Marks	

Component B: Recorded Video - Face + Screen (15 Marks)

Record a video showing your face (webcam picture-in-picture) and your computer's entire screen simultaneously. Explain concepts verbally throughout — do not just click without speaking.

Video Requirements:

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Entire screen (by demonstrating ipynb and pkl file)
Duration	5 to 10 minutes
Naming convention	Practical_Exam_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (5 Marks)

Create a public GitHub repository specifically for this project. Add the following files as mentioned in the Deliverables section below.

📄 Deliverables

Notebook	FraudDetection_SupervisedLearning.ipynb (fully executed, all outputs visible)
Saved Model	fraud_detection_model.pkl (sklearn Pipeline)
Summary Report	summary_report.md (theory + cost-benefit + observations, ~1 page)
GitHub Repo	All of the above + requirements.txt + README.md

✅ **Submission Checklist:**

- ☐ Notebook runs top-to-bottom without errors
- ☐ Test set never resampled — only training data was modified
- ☐ PR-AUC used as primary metric (not ROC-AUC)
- ☐ Threshold optimisation demonstrated with comparison table (Step 6)
- ☐ Cost-benefit table completed with INR values (Step 7)
- ☐ GitHub repo link shared with examiner

BRING ON YOUR CODING ATTITUDE

Practical Exam — Supervised Learning